

COMPTE RENDU

Cloud Foundations & Virtualization — TP2 — AI PaaS Services : Azure
AI Vision, Microsoft Foundry et appels REST
4e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. KARRAKCHOU

Date : 30 septembre 2026

Table des matières

Cadre du TP	2
1 Préparer l'environnement : groupe de ressources, stockage et SAS	3
1.1 Partie I — Création du groupe de ressources	3
1.2 Partie II — Compte de stockage, conteneur Blob et dépôt d'une image	3
1.3 Partie III — Génération d'une URL temporaire (SAS)	5
2 Analyser des images avec Azure AI Vision (REST)	7
2.1 Partie IV — Création de la ressource Computer Vision	7
2.2 Partie V — Analyse d'image avec Postman (tags)	9
2.3 Partie VI — Détection de texte (OCR)	11
2.4 Partie VII — Combiner plusieurs fonctionnalités de Vision	14
3 Interagir avec un LLM via Microsoft Foundry (REST)	16
3.1 Partie VIII — Déploiement d'un modèle	16
3.2 Partie IX — Appel du LLM avec Postman	18
3.3 Partie X — Conversation et mémoire	20
4 Automatiser les appels REST avec Python	23
4.1 Principe et sécurité des identifiants	23
4.2 Script <code>vision.py</code> : Azure AI Vision	24
4.3 Script <code>llm.py</code> : LLM Foundry	26
5 Supprimer les ressources Azure	31
6 Synthèse : utiliser un service d'IA PaaS par appels REST	31
7 Bilan des ressources créées	32
8 Difficultés rencontrées	32

Cadre du TP

Niveau	Cours	Plateformes
4e CY	Cloud Foundations	Azure for Students / Postman / Python
Mode	Approche	Livrable
Groupe de 2 à 3	Labs guidés + appels REST	Compte rendu + captures + code Python

TABLE 1 – Cadre général du TP

Objectif. Découvrir les services d’IA proposés en **PaaS** sur Microsoft Azure et apprendre à les consommer par **appels REST** : analyse d’images et reconnaissance de texte (OCR) avec **Azure AI Vision**, puis dialogue avec un **grand modèle de langage (LLM)** déployé via **Microsoft Foundry**. Les appels sont d’abord réalisés à la main avec Postman, puis automatisés par des programmes Python.

Fil conducteur du TP :

Groupe de ressources → **Blob Storage** + **SAS** → **AI Vision (REST)** → **LLM Foundry (REST)** → **Python**

Environnement. Compte **Azure for Students**, groupe de ressources **TP2-Cloud-Foundation**, région **Sweden Central** (les régions autorisées par la stratégie de l’abonnement sont Poland Central, Spain Central, Austria East, Switzerland North et Sweden Central). Postman pour les appels manuels, Python 3 avec la bibliothèque **requests** pour les scripts, sous Windows (PowerShell).

1 Préparer l'environnement : groupe de ressources, stockage et SAS

Parties I à III du TP : création du groupe de ressources, du compte de stockage et du conteneur Blob, dépôt d'une image, génération d'une URL SAS.

1.1 Partie I — Création du groupe de ressources

Un **groupe de ressources** est un conteneur logique qui regroupe les ressources Azure d'un même projet : on peut les organiser, gérer leurs droits et leurs coûts ensemble et, surtout, les supprimer toutes en une seule opération à la fin du TP.

Créer un groupe de ressources

Informations de base Balises Vérifier + créer

Groupe de ressources - Conteneur qui contient les ressources associées à une solution Azure. Le groupe de ressources peut inclure toutes les ressources de la solution, ou uniquement les ressources que vous voulez gérer en tant que groupe. Vous choisissez la façon dont vous voulez allouer des ressources aux groupes de ressources en fonction de ce qui est le plus adapté à votre organisation. [En savoir plus](#)

Abonnement * Azure for Students

Nom du groupe de ressources * TP2-Cloud-Foundation

Région * (Europe) Sweden Central

FIGURE 1 – « Créer un groupe de ressources » — abonnement Azure for Students, nom TP2-Cloud-Foundation, région Sweden Central.

Paramètre	Valeur retenue
Abonnement	Azure for Students
Nom du groupe de ressources	TP2-Cloud-Foundation
Région	Sweden Central

TABLE 2 – Configuration du groupe de ressources (Partie I)

Remarque sur l'énoncé. L'énoncé nomme le groupe TP2-Cloud-Foundations ; le nom effectivement créé est TP2-Cloud-Foundation (sans « s » final). Cela ne change rien au principe : toutes les ressources du TP sont placées dans ce même groupe.

1.2 Partie II — Compte de stockage, conteneur Blob et dépôt d'une image

La plupart des services d'IA ont besoin de données à traiter ; pour la vision par ordinateur, l'entrée est une image. Dans une architecture cloud classique, l'image est d'abord déposée dans un stockage objet, puis le service d'IA vient la lire via une URL. On crée donc un compte de stockage et un conteneur Blob.

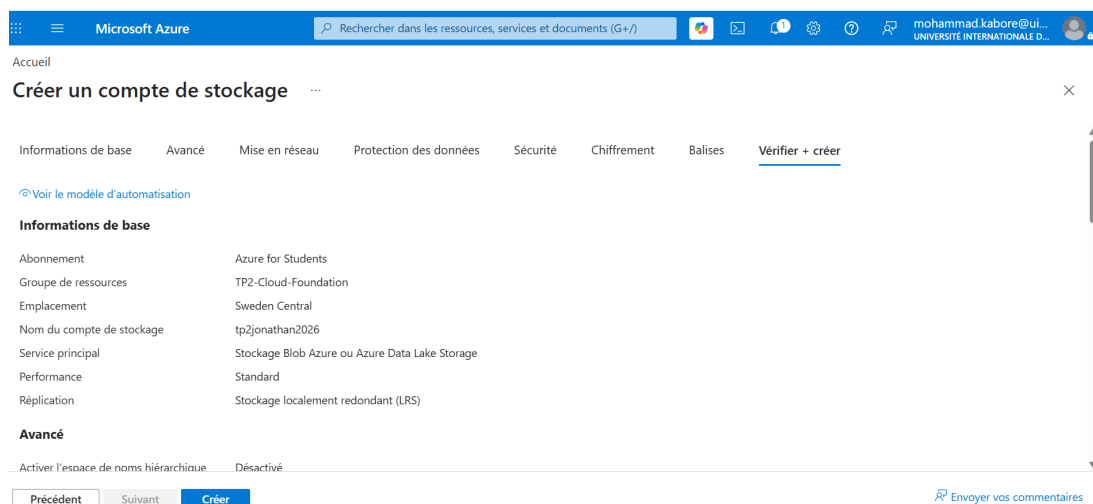


FIGURE 2 – « Créer un compte de stockage » — onglet Vérifier + créer : groupe TP2-Cloud-Foundation, nom tp2jonathan2026, Sweden Central, Standard, LRS.

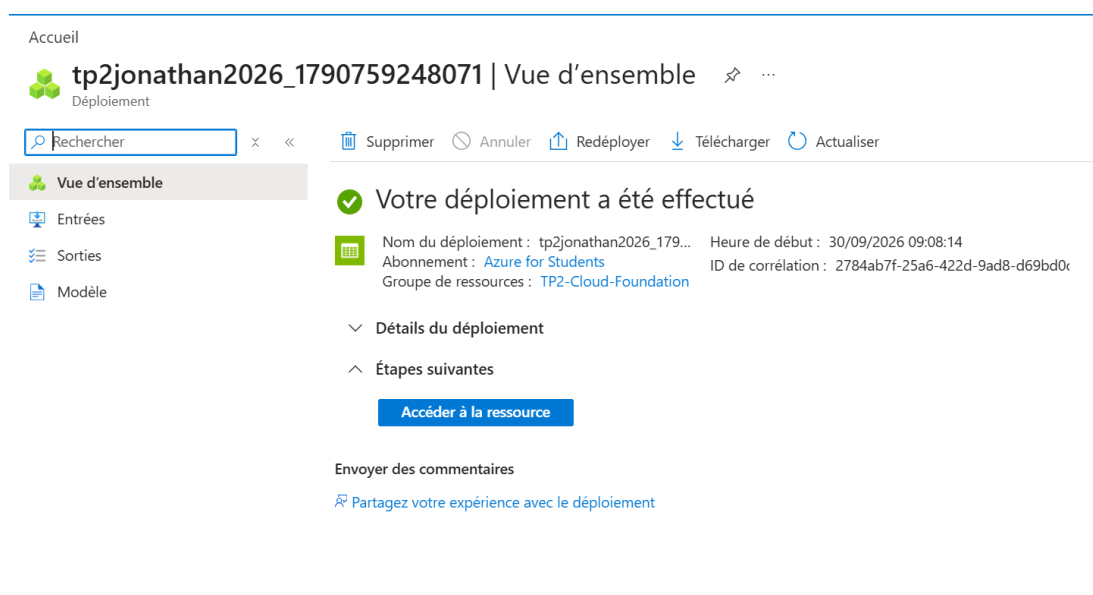


FIGURE 3 – Déploiement effectué : le compte de stockage tp2jonathan2026 est créé dans le groupe de ressources.

Paramètre	Valeur retenue
Groupe de ressources	TP2-Cloud-Foundation
Nom du compte de stockage	tp2jonathan2026 (nom globalement unique)
Région	Sweden Central
Service principal	Stockage Blob Azure ou Azure Data Lake Storage
Performance	Standard
Redondance	Stockage localement redondant (LRS)

TABLE 3 – Configuration du compte de stockage (Partie II)

Dans le compte de stockage, on crée ensuite un conteneur nommé **images**, en conser-

vant le niveau d'accès anonyme **privé**, puis on y dépose une image de plat favori (un tajine poulet citron confit).

Nom	Dernière modification	Niveau d'accès anonyme	État du bail
\$logs	30/09/2026 09:08:41	Privé	Disponible
images	30/09/2026 09:10:43	Privé	Disponible

FIGURE 4 – Conteneurs du compte de stockage : \$logs (système) et images, tous deux d'accès anonyme **Privé**.

Nom	Dernière modification	Niveau d'accès	Type d'objet bl...	Taille	État du bail
tajine poulet citr...	30/09/2026 09:15:41	Chaud (déduit)	Objet blob de ...	75.7 KiB	Disponible

FIGURE 5 – Contenu du conteneur images : le blob tajine poulet citron confit_web.jpg (75,7 Kio, niveau d'accès Chaud).

Question. *Pourquoi un compte de stockage et un conteneur Blob ?*

Le service d'IA n'analyse pas un fichier posé sur un poste local : il télécharge l'image depuis une URL. Le stockage Blob est le stockage objet d'Azure, fait pour héberger ce type de fichiers. Le conteneur est laissé **privé** : personne ne peut lire les images sans autorisation explicite.

Question. *Que signifie LRS ?*

Locally-Redundant Storage : les données sont copiées trois fois au sein d'un même centre de données. C'est l'option la moins chère, suffisante pour un TP, mais elle ne protège pas contre la perte complète du centre de données (contrairement à ZRS ou GRS).

1.3 Partie III — Génération d'une URL temporaire (SAS)

Le conteneur étant privé, le service d'IA ne peut pas télécharger l'image directement. Plutôt que de rendre le conteneur public, on génère une **signature d'accès partagé** (SAS) : une URL signée qui donne un accès *temporaire et restreint* à une ressource de stockage.

tajine poulet citron confit_web.jpg

Objet blob

Enregistrer Ignorer Télécharger Actualiser Supprimer

méthode de signature

☒ Clé de compte ☐ Clé de délégation d'utilisateur

Clé de signature ⓘ

Clé 1

Stratégie d'accès stocké

Aucun

Autorisations * ⓘ

Lecture

Date/heure de début et d'expiration ⓘ

Début

30/09/2026 10:04:40

(UTC+01:00) Casablanca

Expiration

30/09/2026 23:19:40

(UTC+01:00) Casablanca

FIGURE 6 – Génération de la SAS sur le blob du tajine : clé de compte (Clé 1), aucune stratégie d'accès stockée, autorisation **Lecture**, début 10 :04 :40, expiration 23 :19 :40 le 30/09/2026 (UTC+01 :00).

Générer une URL et un jeton SAS

Jeton SAS d'objet blob ⓘ

sp=r&st=2026-09-30T09:04:40Z&se=2026-09-30T22:19:40Z&spr=https&sv=2026-02-06&sr=b[masqué]&sig%3D

URL SAP d'objet blob

https://tp2jonathan2026.blob.core.windows.net/images/tajine%20poulet%20citron%20confit_web.jpg?sp=r&st=2026-09-30T09:04:40Z&se=2026-09-30T22:19:40Z&spr=https&sv=2026-02-06&sr=b[masqué]&sig%3D

FIGURE 7 – « Générer une URL et un jeton SAS » : jeton SAS (**sp=r**, HTTPS uniquement) et URL SAS du blob. La partie signature est masquée.

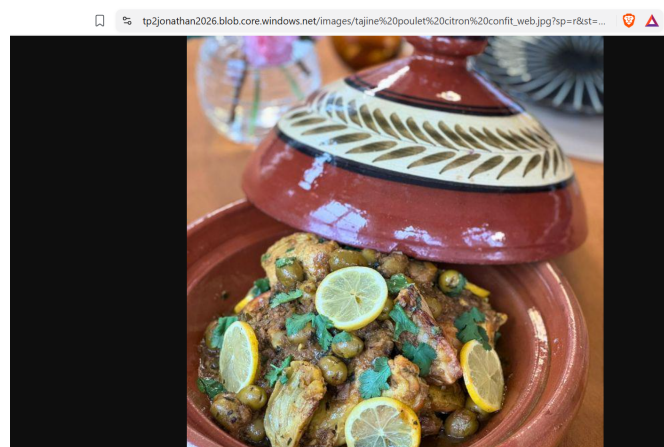


FIGURE 8 – Vérification : l'URL SAS collée dans un nouvel onglet affiche bien l'image du tajine.

Paramètre	Valeur
Méthode de signature	Clé de compte (Clé 1)
Stratégie d'accès stockée	Aucune
Autorisation	Lecture (sp=r)
Protocole	HTTPS uniquement (spr=https)
Validité	30/09/2026, de 10 :04 :40 à 23 :19 :40 (UTC+01 :00)

TABLE 4 – Paramètres de la SAS du tajine (Partie III)

Question. *Qu'est-ce qu'une SAS et pourquoi l'utiliser ?*

Une SAS est une URL signée donnant un accès temporaire et limité à une ressource de stockage. Ici elle n'autorise que la **lecture**, en HTTPS, pendant quelques heures : c'est le principe du **moindre privilège**. Le service Vision peut ainsi télécharger l'image sans que le conteneur devienne public. La partie située après le ? contient la signature : c'est un **secret** qu'il ne faut pas partager (elle est masquée sur toutes les captures).

2 Analyser des images avec Azure AI Vision (REST)

Parties IV à VII du TP : création de la ressource Computer Vision, récupération des identifiants, analyse d'image (tags), détection de texte (OCR), combinaison de plusieurs fonctionnalités.

2.1 Partie IV — Création de la ressource Computer Vision

Azure Computer Vision propose plusieurs fonctionnalités : étiquetage d'images (*tags*), description (*caption*), détection d'objets et de personnes, reconnaissance optique de caractères (OCR).

The screenshot shows the 'Créer une vision par ordinateur' (Create computer vision) page in the Azure portal, specifically the 'Vérifier + créer' (Verify + create) step. The page has a breadcrumb trail: 'Accueil > Microsoft Foundry | Vision par ordinateur'. Below the title, there are tabs for 'Informations de base', 'Réseau', 'Identity', 'Balises', and 'Vérifier + créer'. A link 'View automation template' is visible. The 'Informations de base' (Basic information) section contains the following details:

Informations de base	
Abonnement	Azure for Students
Groupe de ressources	TP2-Cloud-Foundation
Région	Sweden Central
Nom	visison-jonathan-tp2
Niveau tarifaire	Free F0 (20 Calls per minute, 5K Calls per month)

The 'Réseau' (Network) section shows 'Type' as 'Tous les réseaux, y compris Internet, peuvent accéder à cette ressource.' (All networks, including Internet, can access this resource).

At the bottom, there are three buttons: 'Previous' (disabled), 'Suivant' (disabled), and 'Créer' (active).

FIGURE 9 – « Créer une vision par ordinateur » — Vérifier + créer : groupe TP2-Cloud-Foundation, Sweden Central, nom visison-jonathan-tp2, niveau tarifaire gratuit **Free F0**.

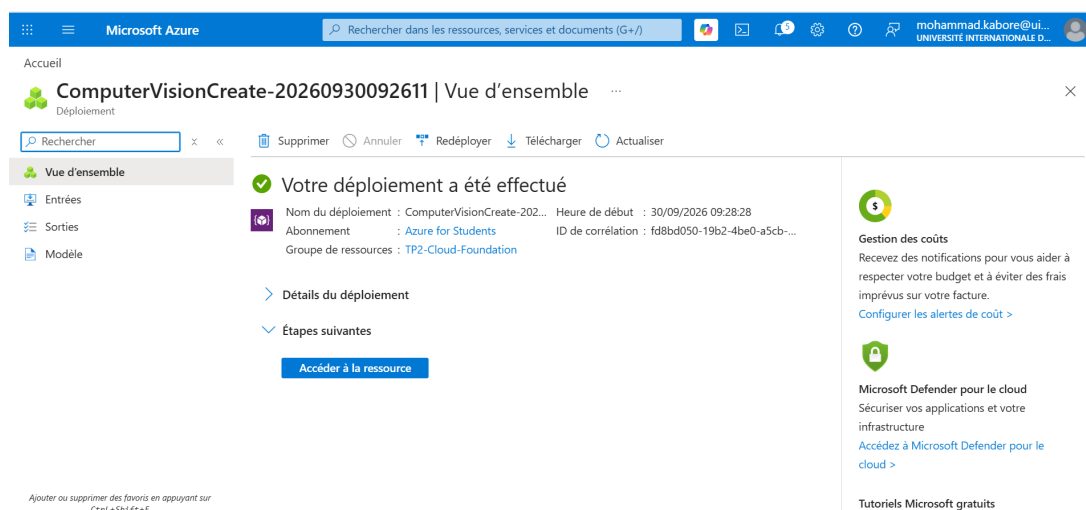


FIGURE 10 – Déploiement ComputerVisionCreate-20260930092611 effectué dans le groupe de ressources.

Paramètre	Valeur retenue
Groupe de ressources	TP2-Cloud-Foundation
Nom	visison-jonathan-tp2
Région	Sweden Central
Niveau tarifaire	Free F0 (20 appels par minute, 5 000 appels par mois)
Réseau	Tous les réseaux, y compris Internet

TABLE 5 – Configuration de la ressource Computer Vision (Partie IV)

Récupération des identifiants d'API

Une API REST exige une authentification. Dans la section « Clés et point de terminaison » de la ressource, on trouve **deux clés** (KEY 1, KEY 2) et le **point de terminaison** (*endpoint*) : <https://visison-jonathan-tp2.cognitiveservices.azure.com/>.

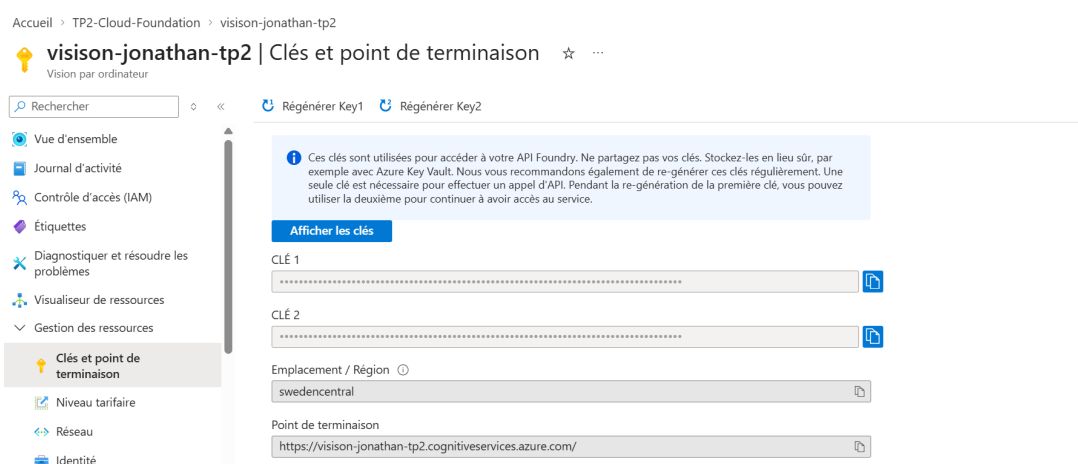


FIGURE 11 – Section « Clés et point de terminaison » de la ressource visison-jonathan-tp2 (région swedencentral). Les clés restent masquées.

Les deux clés permettent une **rotation sans coupure** : on bascule les applications sur la clé 2, puis on régénère la clé 1. Une clé d'API est un **secret** : elle ne doit jamais être publiée ni versée dans un dépôt GitHub public.

2.2 Partie V — Analyse d'image avec Postman (tags)

On appelle Azure AI Vision par une requête HTTP **POST**. L'URL utilisée est l'endpoint suivi du chemin de l'API Image Analysis, avec la fonctionnalité **tags** :

Listing 1 – Requête d'analyse d'image (tags)

```
1 POST https://visison-jonathan-tp2.cognitiveservices.azure.com/
  computervision/imageanalysis:analyze?api-version=2024-02-01&
  features=tags
2
3 Headers :
4   Ocp-Apim-Subscription-Key: <CLE_API>
5   Content-Type: application/json
6
7 Body (raw, JSON) :
8 {
9   "url": "<URL_SAS_DU_TAJINE>"
10 }
```

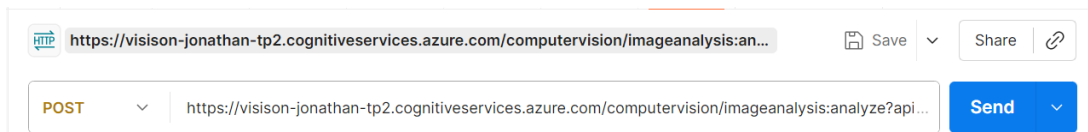


FIGURE 12 – Requête Postman : méthode POST vers l'endpoint **imageanalysis:analyze** avec **features=tags**.

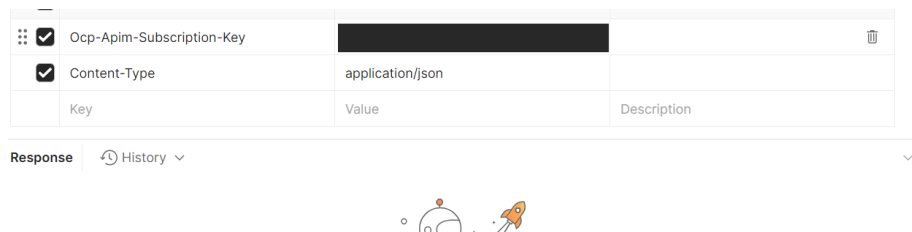


FIGURE 13 – En-têtes de la requête : **Ocp-Apim-Subscription-Key** (valeur masquée) et **Content-Type: application/json**.

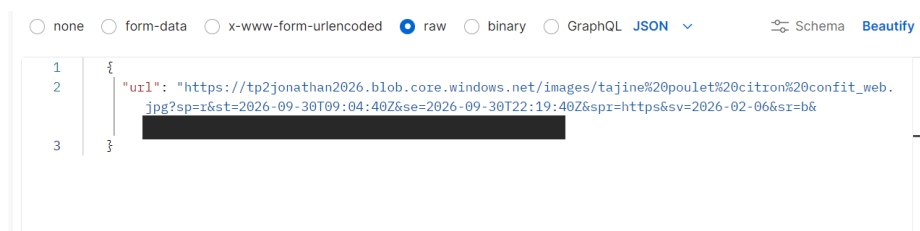


FIGURE 14 – Corps de la requête : l'URL SAS de l'image (signature masquée).

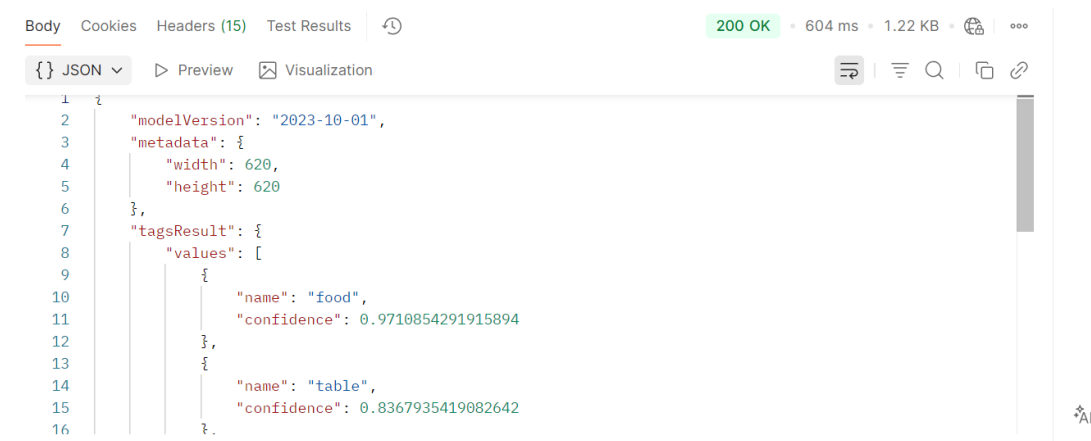


FIGURE 15 – Réponse **200 OK** (604 ms) : modelVersion 2023-10-01, image de 620 × 620 pixels, début de tagsResult (food, table).

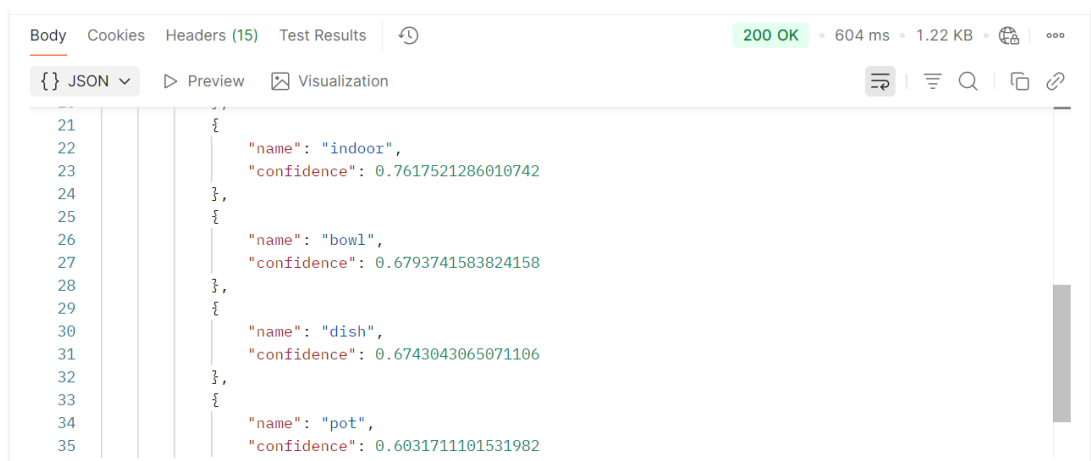


FIGURE 16 – Suite de la réponse : tags indoor, bowl, dish et pot.

Tag	Confiance
food	0,971
table	0,837
plant	0,832
indoor	0,762
bowl	0,679
dish	0,674
pot	0,603

TABLE 6 – Tags détectés sur l'image du tajine (Partie V)

Questions.

1. *Quels tags ont été détectés ?* Sept tags : food, table, plant, indoor, bowl, dish et pot.
2. *Quel tag a la plus haute confiance ?* food, avec environ 0,97 (97 %).

3. *Que représente la valeur de confiance ?* Un score entre 0 et 1 exprimant la probabilité, estimée par le modèle, que le tag soit correct. Plus il est proche de 1, plus le modèle est sûr de lui ; ce n'est pas une garantie de justesse, seulement une mesure de certitude du modèle.
4. *Le service a-t-il correctement identifié l'image ?* Partiellement. Il reconnaît de la nourriture dans un récipient (**bowl**, **dish**, **pot**) sur une table, en intérieur, ainsi que le vase fleuri à l'arrière-plan (**plant**). En revanche, il n'identifie ni le tajine ni ses ingrédients (poulet, citron, olives) : le couvercle en terre cuite est vu comme un simple « pot ». Le modèle est fiable sur les catégories générales, moins sur un plat culturellement spécifique.

2.3 Partie VI — Détection de texte (OCR)

L'OCR consiste à détecter et extraire le texte contenu dans une image. On utilise une image comportant du texte bien visible : la couverture du livre *Les Bouts de bois de Dieu* d'Ousmane Sembène (éditions Pocket). Elle est déposée dans le conteneur **images**, puis on génère une SAS en lecture seule.

Affichage de tous les éléments 2

☐	Nom	Dernière modification	Niveau d'accès	Type d'objet bl...	Taille	État du bail	
☐	 couverture_les_b...	30/09/2026 10:14:10	Chaud (déduit)	Objet blob de ...	33.97 KiB	Disponible	...
☐	 tajine poulet citr...	30/09/2026 09:15:41	Chaud (déduit)	Objet blob de ...	75.7 KiB	Disponible	...

FIGURE 17 – Conteneur **images** : deux blobs, la couverture du livre (33,97 Kio) et le tajine (75,7 Kio).

couverture_les_bouts_de_bois_de_Dieu.jpg

Objet blob

 Enregistrer
  Ignorer
  Télécharger
  Actualiser
  Supprimer

Méthode de signature

☒ Clé de compte
 ☐ Clé de délégation d'utilisateur

Clé de signature 

Clé 1 

Stratégie d'accès stocké

Aucun 

Autorisations * 

Lecture 

Date/heure de début et d'expiration 

Début

30/09/2026 

11:12:31

(UTC+01:00) Casablanca 

Expiration

30/09/2026 

23:27:31

(UTC+01:00) Casablanca 

FIGURE 18 – SAS de la couverture : clé de compte, autorisation **Lecture**, début 11 :12 :31, expiration 23 :27 :31 le 30/09/2026.



FIGURE 19 – Vérification : l’URL SAS de la couverture s’ouvre correctement dans le navigateur.

La requête est identique à la précédente, avec la fonctionnalité `read` et l’URL SAS de la couverture :

Listing 2 – Requête OCR

```
1 POST https://visison-jonathan-tp2.cognitiveservices.azure.com/
  computervision/imageanalysis:analyze?api-version=2024-02-01&
  features=read
2
3 { "url": "<URL_SAS_DE_LA_COUVERTURE>" }
```

```
7   "readResult": {
8     "blocks": [
9       {
10        "lines": [
11          {
12            "text": "OUSMANE",
13            "boundingPolygon": [
14              {
15                "x": 75,
16                "y": 20
17              },
18              {
19                "x": 223,
20                "y": 19
21              }
22            ]
23          }
24        ]
25      }
26    ]
27  }
```

FIGURE 20 – Réponse **200 OK** (1,51 s) : structure `readResult` → `blocks` → `lines`, première ligne « OUSMANE ».

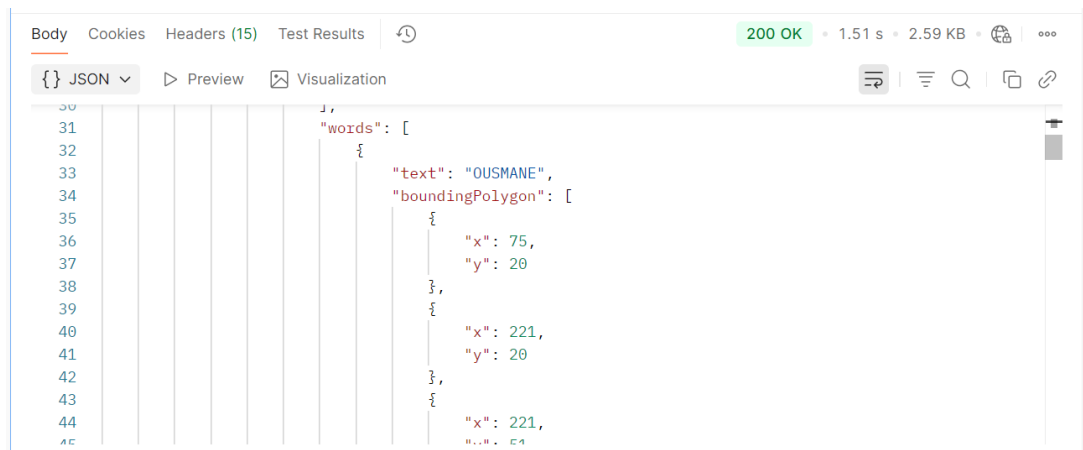


FIGURE 21 – Détail d’une ligne : la liste **words** contient le mot « OUSMANE » avec son **boundingPolygon**.

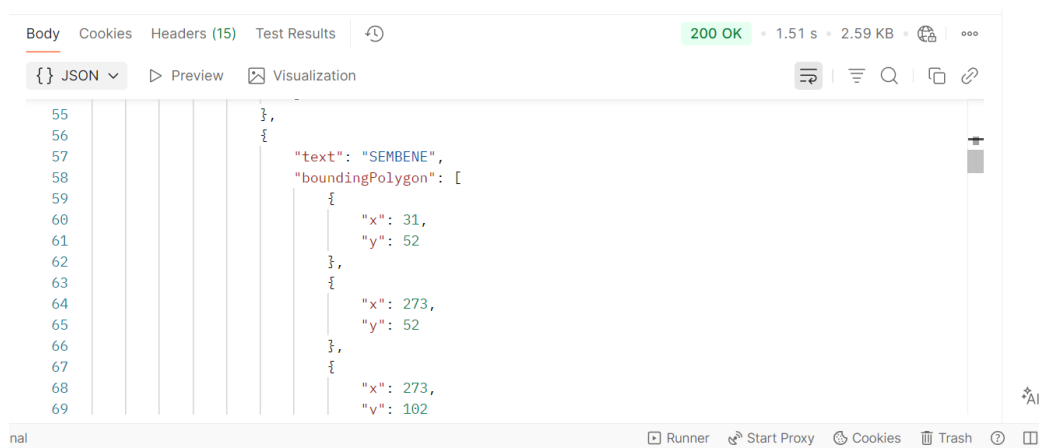


FIGURE 22 – Ligne « SEMBENE » et ses coordonnées.

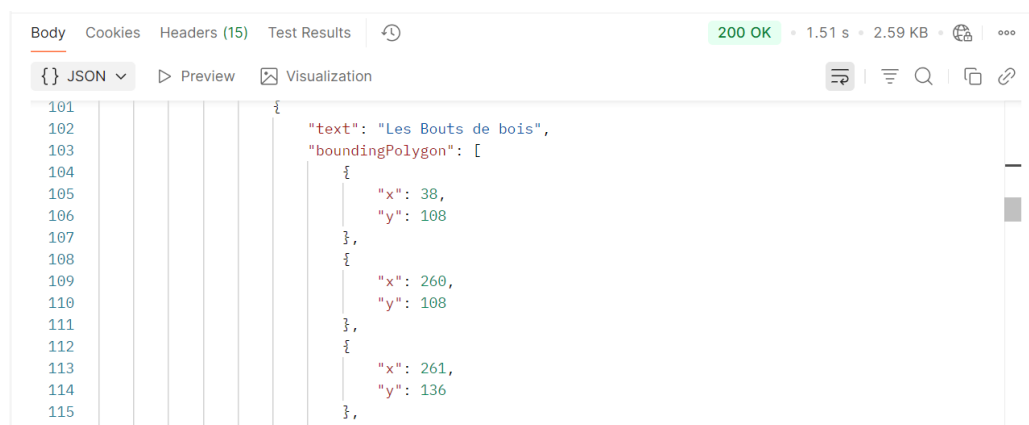


FIGURE 23 – Ligne « Les Bouts de bois » et ses coordonnées.



FIGURE 24 – Ligne « POCKET » : le texte est vertical, le long du bord gauche de la couverture, et a pourtant été lu.

Ligne détectée	Étendue en x (px)	Étendue en y (px)
OUSMANE	75 – 224	19 – 52
SEMBENE	31 – 273	52 – 102
Les Bouts de bois	38 – 261	108 – 136
de Dieu	99 – 203	137 – 163
POCKET	14 – 32	406 – 473

TABLE 7 – Texte détecté sur la couverture et position approximative (Partie VI)

Questions.

1. *Quel texte a été détecté ?* Cinq lignes : « OUSMANE », « SEMBENE », « Les Bouts de bois », « de Dieu » et « POCKET ».
2. *Comment les lignes sont-elles représentées dans le JSON ?* Par une structure hiérarchique `readResult` → `blocks` → `lines` → `words`. Chaque ligne a un `text`, un `boundingPolygon` et une liste de mots ; chaque mot a son texte, son polygone et sa `confidence`. Le titre est coupé en deux lignes, comme sur la couverture : le modèle suit la mise en page visuelle, pas le sens.
3. *Azure donne-t-il la position du texte ?* Oui : chaque ligne et chaque mot possèdent un `boundingPolygon` de quatre points (x, y) en pixels, l'origine étant en haut à gauche de l'image (302×500 px). Un polygone plutôt qu'un rectangle permet de décrire du texte incliné ou pivoté, comme « POCKET » écrit verticalement.
4. *Tout le texte a-t-il été correctement reconnu ?* Oui, les cinq éléments sont reconnus sans erreur, avec des confiances entre 0,992 et 0,998, malgré des polices de tailles différentes, un fond texturé et un texte vertical.

2.4 Partie VII — Combiner plusieurs fonctionnalités de Vision

Azure AI Vision peut demander plusieurs fonctionnalités dans le même appel. Le TP propose de modifier l'URL en `features=tags,caption`. Avec l'API 4.0, cette requête a renvoyé une **erreur 400** : « The feature 'Caption' is not supported in this region ». La `caption` de l'API 4.0 n'est disponible que dans certaines régions (East US, France Central, West Europe, North Europe...), et aucune des cinq régions autorisées par l'abonnement ne la propose.

Solution : utiliser l'**API Image Analysis 3.2**, dont la fonctionnalité *Description* est disponible dans toutes les régions.

Listing 3 – Requête tags + description (API 3.2)

```
1 POST https://visison-jonathan-tp2.cognitiveservices.azure.com/  
vision/v3.2/analyze?visualFeatures=Tags,Description  
2  
3 { "url": "<URL_SAS_DU_TAJINE>" }
```

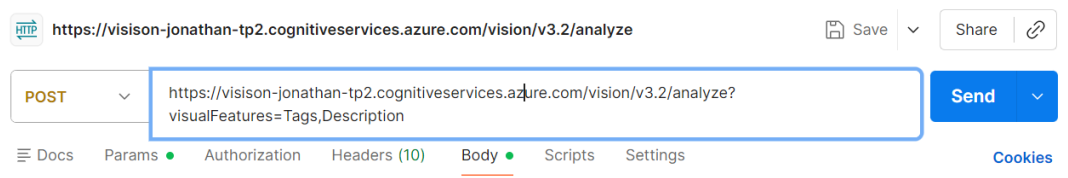


FIGURE 25 – Requête Postman vers l'API 3.2 (/vision/v3.2/analyze) avec visualFeatures=Tags,Description.

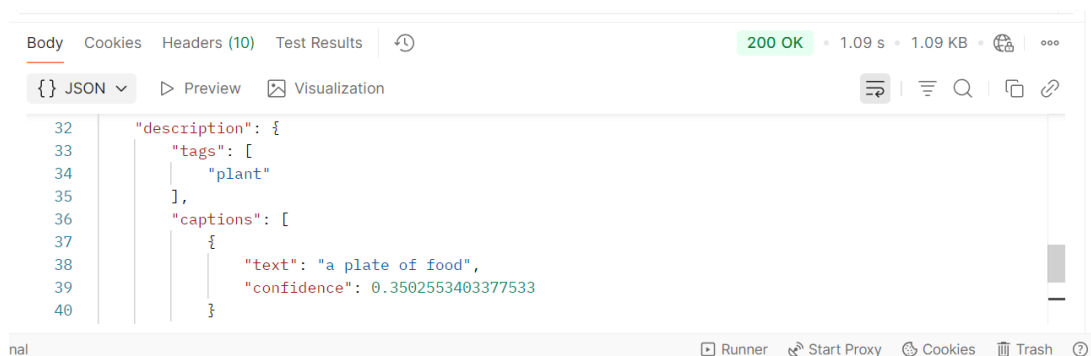


FIGURE 26 – Réponse **200 OK** (1,09 s) : dans **description**, la caption "a plate of food" avec une confiance de 0,350.

Questions.

1. *Différence entre un tag et une caption ?* Un tag est un mot-clé isolé désignant un élément détecté, avec un score de confiance. Une caption est une phrase complète en langage naturel, générée par le modèle, qui décrit l'image dans son ensemble.
2. *Combien de tags ont été retournés ?* Sept tags, identiques (mêmes noms et mêmes scores) à ceux de la Partie V.
3. *Quelle caption a été générée ?* « a plate of food » (une assiette de nourriture).
4. *Quelle confiance est associée à la caption ?* Environ 0,35 (35 %), bien plus faible que celle des principaux tags.
5. *La description est-elle correcte ?* Correcte mais très générique : c'est bien de la nourriture, mais ce n'est pas une assiette, c'est un tajine dans un plat en terre cuite avec son couvercle. La faible confiance montre que le modèle est lui-même peu sûr ; le modèle de l'API 3.2 est plus ancien et moins performant que celui de l'API 4.0, indisponible dans la région.

3 Interagir avec un LLM via Microsoft Foundry (REST)

Parties VIII à X du TP : création de la ressource Foundry, déploiement d'un modèle, appel par l'API Chat Completions avec Postman, mémoire de conversation.

3.1 Partie VIII — Déploiement d'un modèle

Les grands modèles de langage (LLM) sont des modèles d'IA générative capables de traiter le langage naturel : répondre à des questions, résumer, traduire, générer du code, extraire de l'information. Azure y donne accès via **Microsoft Foundry**.

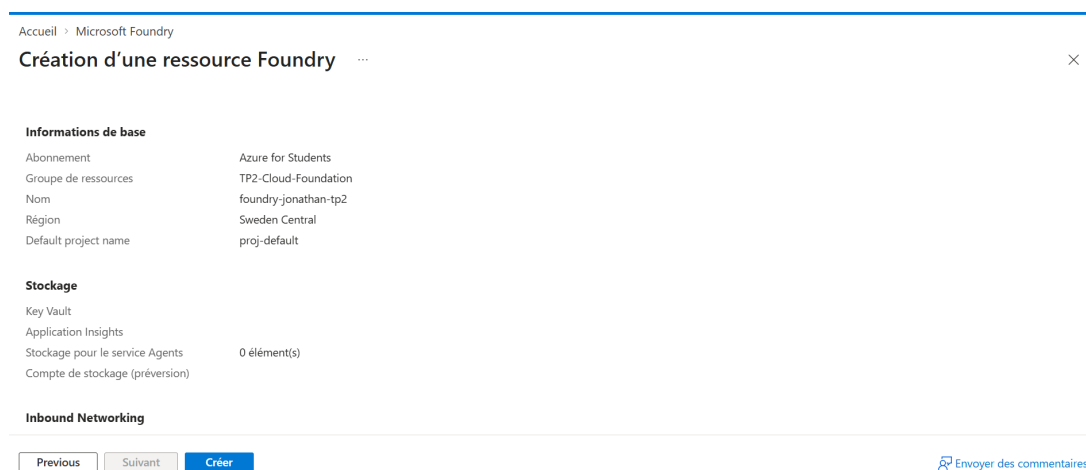


FIGURE 27 – « Création d'une ressource Foundry » : groupe TP2-Cloud-Foundation, nom foundry-jonathan-tp2, Sweden Central, projet par défaut proj-default.

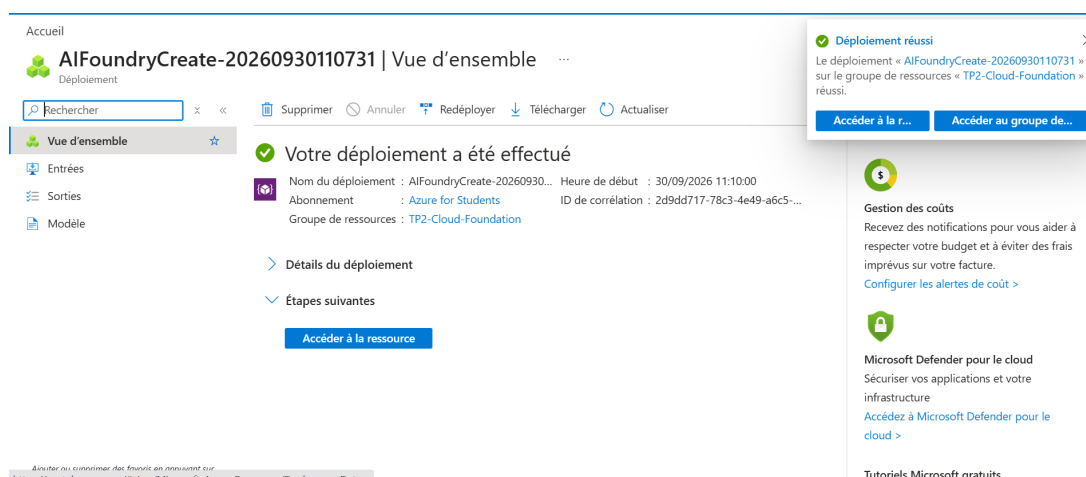


FIGURE 28 – Déploiement AIFoundryCreate-20260930110731 effectué dans le groupe de ressources.

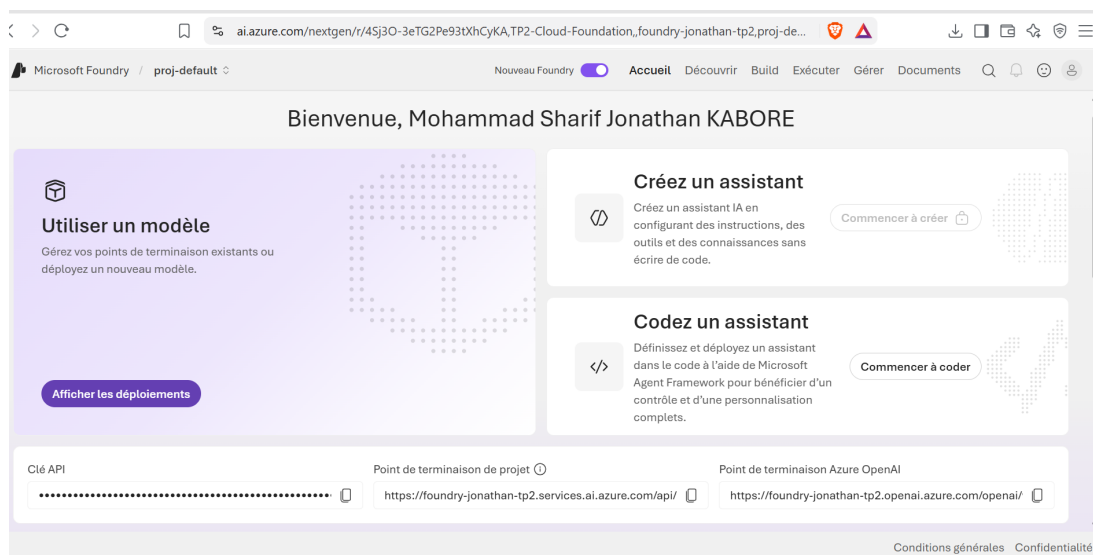


FIGURE 29 – Accueil du projet Foundry : clé d’API (masquée), point de terminaison du projet et point de terminaison Azure OpenAI.

Le modèle proposé par l’énoncé, **gpt-5.4-nano**, n’a pas pu être déployé (quota insuffisant), et **gpt-4o-mini** a été refusé car déprécié (voir la section « Difficultés rencontrées »). Le modèle retenu est **gpt-5-mini**, déployé en *Global Standard* avec le filtre de contenu par défaut.

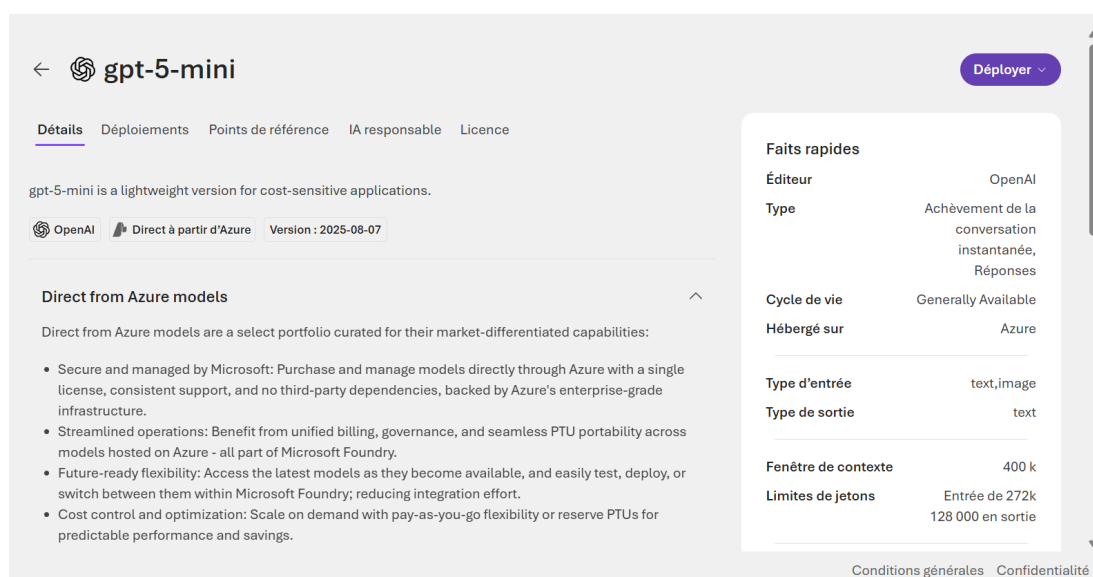


FIGURE 30 – Fiche du modèle **gpt-5-mini** (OpenAI, version 2025-08-07) : entrée texte et image, sortie texte, fenêtre de contexte de 400 k tokens.

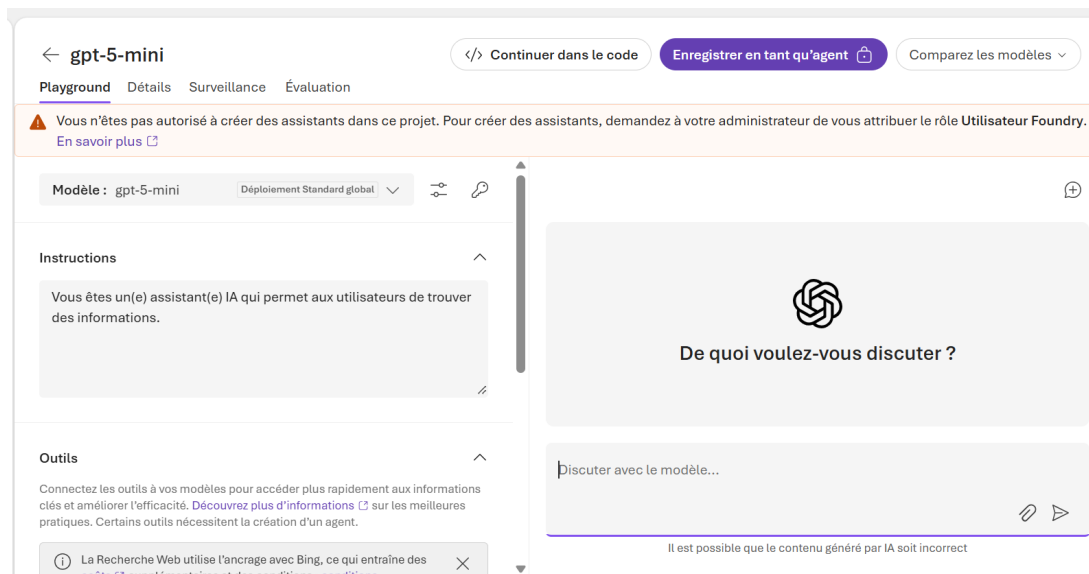


FIGURE 31 – Playground du modèle gpt-5-mini (déploiement Standard global) prêt à recevoir des messages.

Paramètre	Valeur
Ressource Foundry	foundry-jonathan-tp2 (projet proj-default)
Région	Sweden Central
Modèle déployé	gpt-5-mini (version 2025-08-07)
Type de déploiement	Standard global (Global Standard)
Fenêtre de contexte	400 k tokens (entrée 272 k, sortie 128 k)

TABLE 8 – Déploiement du modèle (Partie VIII)

3.2 Partie IX — Appel du LLM avec Postman

On envoie une requête POST à l'API **Chat Completions**. La clé d'API est placée dans l'onglet **Authorization**, avec le type **Bearer Token**, et l'en-tête **Content-Type: application/json** est ajouté.

Listing 4 – Requête Chat Completions

```

1 POST https://foundry-jonathan-tp2.services.ai.azure.com/openai/v1/
  chat/completions
2
3 Authorization: Bearer <CLE_API>
4 Content-Type: application/json
5
6 {
7   "messages": [
8     { "role": "user", "content": "Explain cloud computing in three
      sentences." }
9   ],
10  "model": "gpt-5-mini",
11  "max_completion_tokens": 4096,
12  "reasoning_effort": "medium"

```

L'endpoint affiché par défaut dans Foundry pointait vers la *Responses API* (/responses), qui attend un champ `input` au lieu de `messages` (erreur « `Unsupported parameter : messages` ») : on utilise donc le chemin `/openai/v1/chat/completions`.

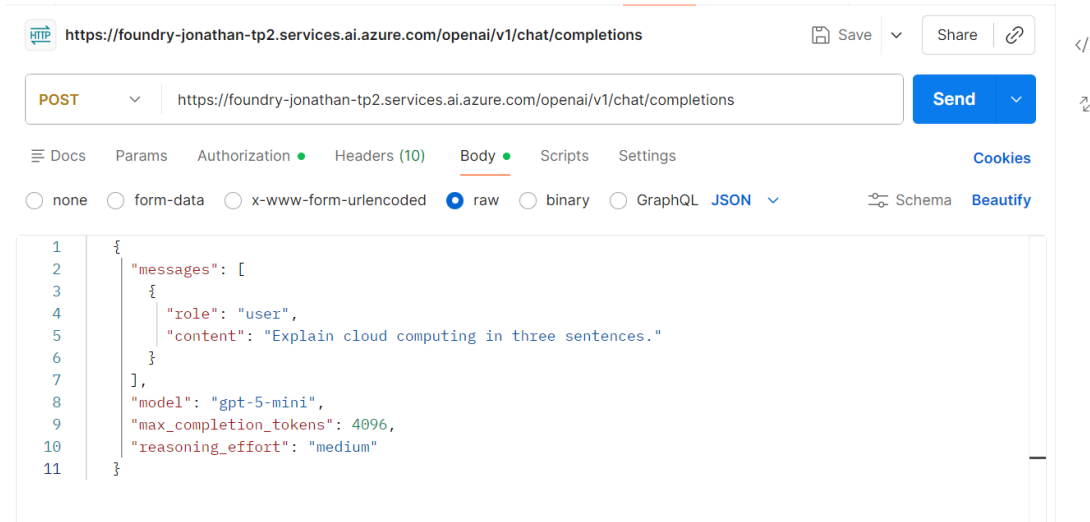


FIGURE 32 – Requête Postman : POST vers `.../openai/v1/chat/completions`, corps JSON avec le modèle `gpt-5-mini`.

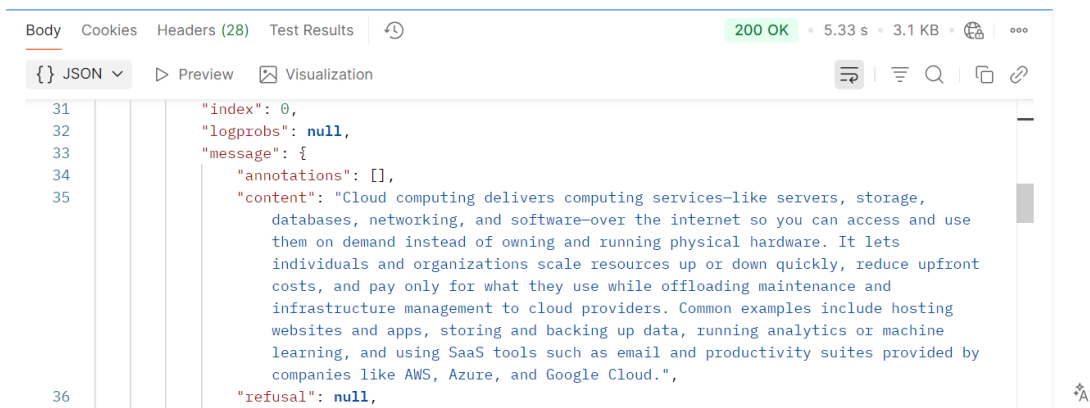


FIGURE 33 – Réponse **200 OK** (5,33 s) : le texte généré se trouve dans `choices[0].message.content`.

```

72   "routing": {
73     "serving_pipereplica": "d20260928233523-7bdf26db-default-r2-dp0-default"
74   },
75   "service_tier": "default",
76   "system_fingerprint": null,
77   "usage": {
78     "completion_tokens": 316,
79     "completion_tokens_details": {
80       "accepted_prediction_tokens": 0,
81       "audio_tokens": 0,
82       "reasoning_tokens": 192,
83       "rejected_prediction_tokens": 0
84     },
85     "latency_checkpoint": {

```

FIGURE 34 – Bloc usage : completion_tokens = 316, dont reasoning_tokens = 192.

```

Body Cookies Headers (28) Test Results 200 OK • 5.33 s • 3.1 KB
{} JSON Preview Visualization
89   "pre_inference_ms": 85,
90   "service_tbt_ms": 11,
91   "service_ttft_ms": 585,
92   "service_tttl_ms": 4348,
93   "user_visible_ttft_ms": 500
94 },
95   "prompt_tokens": 13,
96   "prompt_tokens_details": {
97     "audio_tokens": 0,
98     "cached_tokens": 0
99   },
100   "total_tokens": 329
101 }
102 }

```

FIGURE 35 – Fin de la réponse : prompt_tokens = 13, total_tokens = 329, et mesures de latence (service_ttft_ms = 585, service_tttl_ms = 4348).

Questions.

1. Où se trouve le texte généré ? Dans `choices[0].message.content` ; le message porte aussi `role = assistant` et un champ `refusal` à `null` (le modèle n'a pas refusé de répondre).
2. Quelles autres informations sont retournées ? Des métadonnées (`id`, `created`, `model = gpt-5-mini-2025-08-07`, `object`), le `finish_reason` égal à `stop` (fin naturelle de la génération ; on aurait `length` en cas de coupure par la limite de tokens), les résultats des filtres de contenu (question et réponse, tous « safe ») et des mesures de latence (environ 4,3 s pour la réponse complète, 0,5 s avant le premier token).
3. Trouve-t-on le nombre de tokens utilisés ? Oui, dans `usage` : `prompt_tokens = 13`, `completion_tokens = 316`, `total_tokens = 329`. Détail important : `reasoning_tokens = 192`. GPT-5 mini est un modèle de raisonnement : il « réfléchit » avec des tokens invisibles dans la réponse mais comptés et facturés ; seuls environ 124 tokens correspondent au texte visible. Le paramètre `reasoning_effort` règle cette réflexion.

3.3 Partie X — Conversation et mémoire

Expérience 1 : deux requêtes séparées. On envoie d'abord « My favorite programming language is Python. Remember this information. », puis, dans une *nouvelle* requête, seulement « What is my favorite programming language? ».

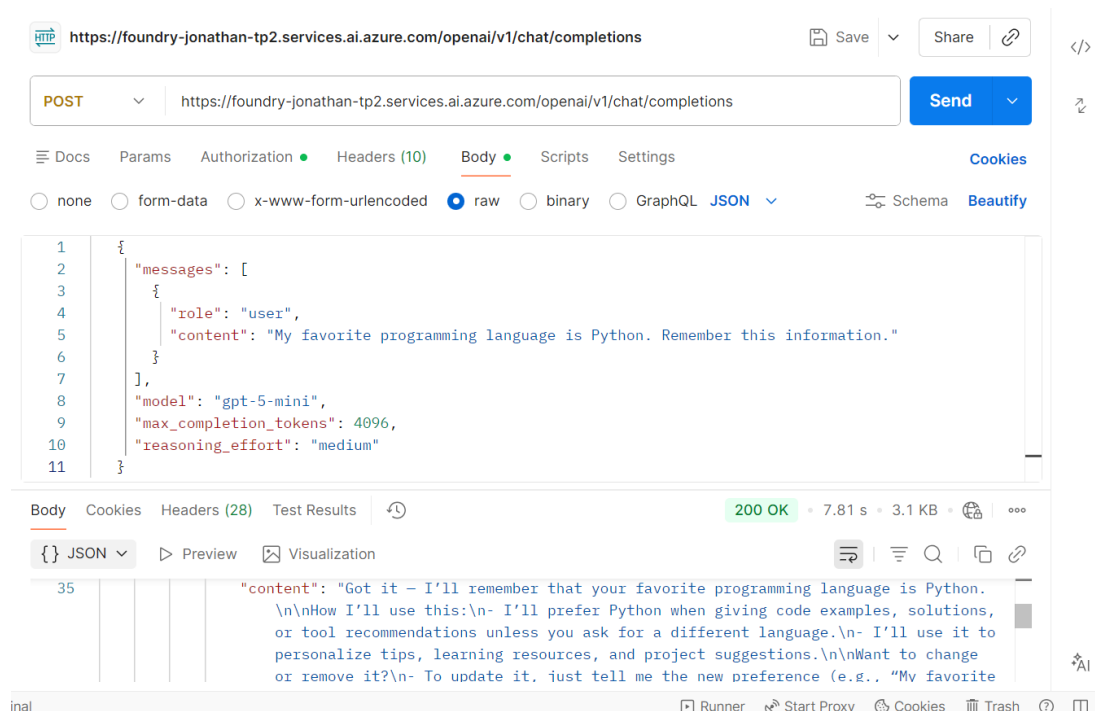


FIGURE 36 – Première requête : le modèle répond « Got it — I'll remember that your favorite programming language is Python ».

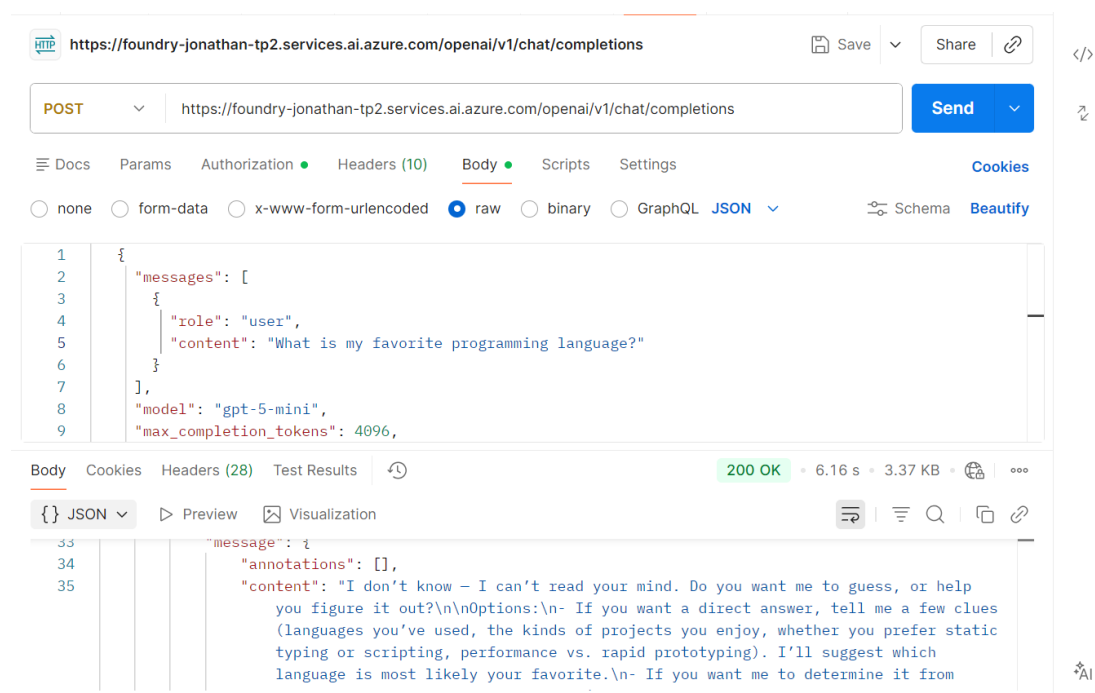


FIGURE 37 – Seconde requête, seule : le modèle répond « I don't know — I can't read your mind ».

Question. *Le modèle se souvient-il nécessairement de la requête précédente ? Pourquoi ?*

Non. L'API REST est **sans état** (*stateless*) : chaque requête HTTP est indépendante

et le serveur ne conserve rien entre deux appels. Le modèle ne voit que les messages présents dans la requête en cours. Au passage, il a affirmé « je m’en souviendrai » alors qu’il en est incapable : un LLM génère une réponse plausible sans connaître ses propres limites techniques.

Expérience 2 : historique envoyé dans une seule requête.

Listing 5 – Conversation complète envoyée en une requête

```
1 {  
2   "messages": [  
3     { "role": "user",      "content": "My favorite programming  
4       language is Python." },  
5     { "role": "assistant", "content": "Understood." },  
6     { "role": "user",      "content": "What is my favorite  
7       programming language?" }  
  ]  
}
```

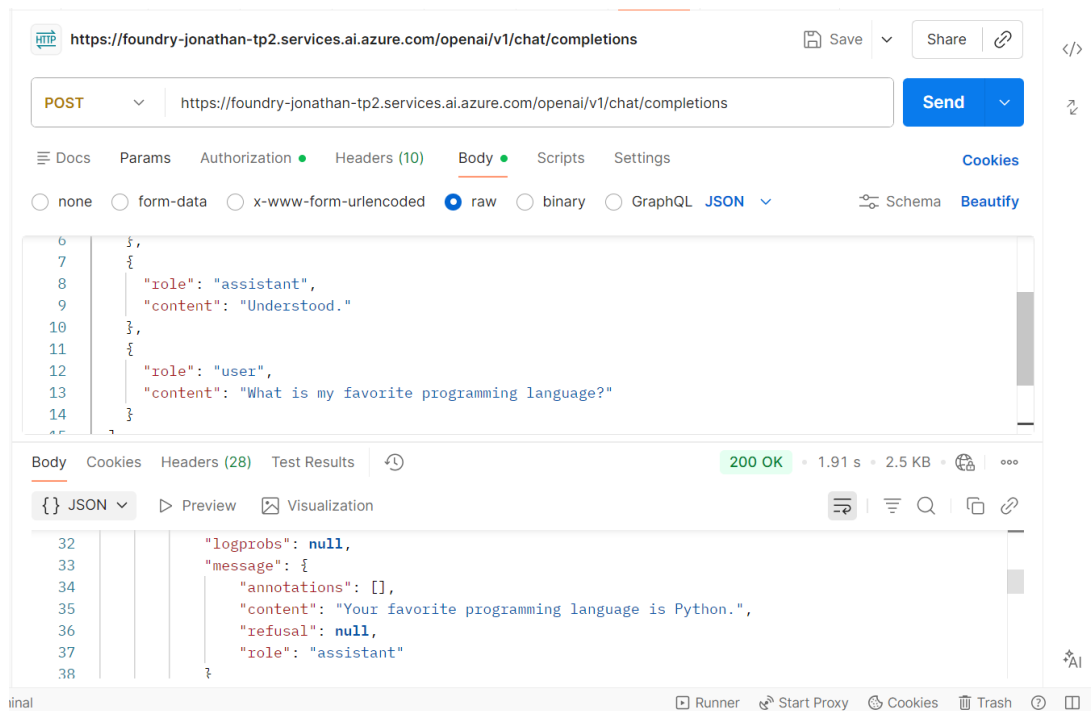


FIGURE 38 – Requête contenant l'historique : le modèle répond « Your favorite programming language is Python. »

Questions.

1. *Différence entre les deux expériences ?* Dans la première, les messages sont envoyés dans deux requêtes séparées : le modèle n'a aucun contexte et ne peut pas répondre. Dans la seconde, une seule requête contient tout l'historique (user, assistant, user) et le modèle répond correctement.
2. *Qui maintient l'historique ?* L'**application cliente** (Postman, puis notre script Python) : elle stocke les échanges et renvoie l'historique dans le tableau `messages` à chaque requête, comme le font les interfaces de type ChatGPT.

3. *Pourquoi renvoyer toute la conversation devient problématique ?* Le **coût** augmente (tout l'historique est recompté en `prompt_tokens` à chaque requête); la **fenêtre de contexte** est limitée (au-delà, il faut tronquer et le modèle « oublie » le début); la **latence** croît avec la taille de la requête; et la **qualité** peut baisser sur un contexte très long. Solutions : ne garder que les derniers messages, résumer l'ancien historique, ou stocker à part les informations importantes.

Conversation personnalisée (trois messages utilisateur). Une conversation de notre cru est envoyée à l'API : « I am from Ouagadougou, Burkina Faso. » puis « I am studying cybersecurity at a university in Rabat. » puis « Where am I from, where do I study, and what is my field ? ».

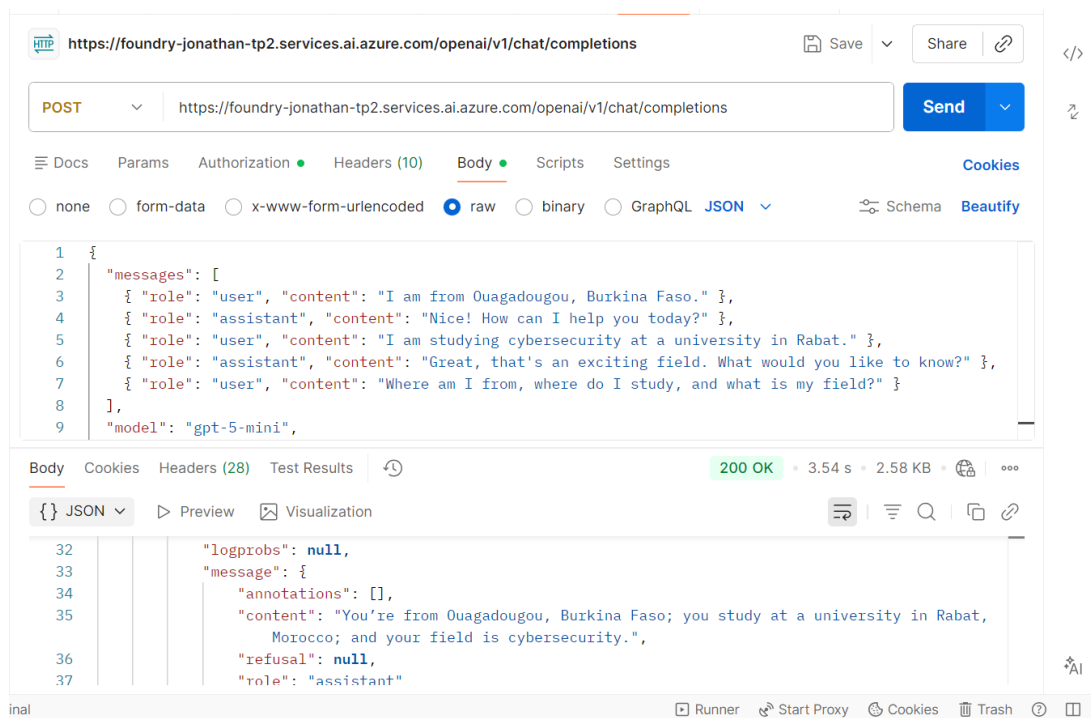


FIGURE 39 – Conversation de trois messages utilisateur : le modèle répond « You're from Ouagadougou, Burkina Faso; you study at a university in Rabat, Morocco; and your field is cybersecurity. »

Le modèle a ajouté de lui-même « Morocco » : il combine le contexte fourni et ses connaissances générales.

4 Automatiser les appels REST avec Python

Partie XI du TP : deux programmes Python, basés sur la bibliothèque `requests`, reproduisent les appels réalisés manuellement avec Postman.

4.1 Principe et sécurité des identifiants

Dans une vraie application cloud, ces requêtes REST sont émises par un programme. Chaque script suit le même schéma : construction de l'URL, des en-têtes (clé d'API) et du corps JSON, envoi par `requests.post(...)`, contrôle du code HTTP (en cas d'erreur,

affichage du message renvoyé par Azure), puis lecture du JSON par `response.json()` et extraction des champs utiles.

Les clés et endpoints ne sont **jamais écrits dans le code** : ils sont lus dans des variables d'environnement, ce qui permet de publier le code sans exposer de secret.

Listing 6 – Définition des variables d'environnement (PowerShell) et exécution

```
1 $env:VISION_ENDPOINT="https://visison-jonathan-tp2.
   cognitiveservices.azure.com"
2 $env:VISION_KEY="<CLE_VISION>"
3 $env:FOUNDRY_ENDPOINT="https://foundry-jonathan-tp2.services.ai.
   azure.com/openai/v1/chat/completions"
4 $env:FOUNDRY_KEY="<CLE_FOUNDRY>"
5
6 python vision.py tags "<URL_SAS>"
7 python llm.py chat
```

4.2 Script vision.py : Azure AI Vision

Le script propose trois modes : `tags` (API 4.0, `features=tags`), `ocr` (API 4.0, `features=read`) et `caption` (API 3.2, `tags + description`).

Listing 7 – Script vision.py — tags, OCR et caption

```
1 """
2 TP2 - Appels REST à Azure AI Vision avec la bibliothèque requests.
3
4 Utilisation :
5 python vision.py tags URL_SAS_IMAGE
6 python vision.py ocr URL_SAS_IMAGE
7 python vision.py caption URL_SAS_IMAGE
8
9 Les secrets ne sont jamais écrits dans le code : ils sont lus
10 depuis les variables d'environnement VISION_ENDPOINT et VISION_KEY.
11 """
12
13 import os
14 import sys
15
16 import requests
17
18 ENDPOINT = os.environ["VISION_ENDPOINT"].rstrip("/")
19 KEY = os.environ["VISION_KEY"]
20
21 HEADERS = {
22     "Ocp-Apim-Subscription-Key": KEY,
23     "Content-Type": "application/json",
24 }
25
26
27 def post(url, params, image_url):
28     """Envoie la requête POST et affiche le message d'erreur Azure
   en cas d'échec."""
```

```

29     response = requests.post(
30         url, headers=HEADERS, params=params, json={"url": image_url
31         }, timeout=30
32     )
33     if not response.ok:
34         print(f"Erreur HTTP {response.status_code}: {response.text}")
35         sys.exit(1)
36     return response.json()
37
38 def analyze_v4(image_url, features):
39     """Image Analysis 4.0 (parties V et VI)."""
40     url = f"{ENDPOINT}/computervision/imageanalysis:analyze"
41     params = {"api-version": "2024-02-01", "features": features}
42     return post(url, params, image_url)
43
44
45 def analyze_v32(image_url):
46     """Image Analysis 3.2: caption disponible dans toutes les régions (partie VII)."""
47     url = f"{ENDPOINT}/vision/v3.2/analyze"
48     params = {"visualFeatures": "Tags,Description"}
49     return post(url, params, image_url)
50
51
52 def show_tags(image_url):
53     data = analyze_v4(image_url, "tags")
54     print("Tags détectés:")
55     for tag in data["tagsResult"]["values"]:
56         print(f"_{tag['name']}:<12}_{tag['confidence']:.3f}")
57
58
59 def show_ocr(image_url):
60     data = analyze_v4(image_url, "read")
61     print("Texte détecté:")
62     for block in data["readResult"]["blocks"]:
63         for line in block["lines"]:
64             points = [(p["x"], p["y"]) for p in line["boundingPolygon"]]
65             print(f"_{line['text']}\_{line['position']:.3f}")
66
67
68 def show_caption(image_url):
69     data = analyze_v32(image_url)
70     print("Tags détectés:")
71     for tag in data["tags"]:
72         print(f"_{tag['name']}:<12}_{tag['confidence']:.3f}")
73     print("Caption:")

```

```

74     for caption in data["description"]["captions"]:
75         print(f"_{caption['text']}_{confidence}_{caption['confidence']:.3f}")
76
77
78 if __name__ == "__main__":
79     actions = {"tags": show_tags, "ocr": show_ocr, "caption":
80               show_caption}
81
82     if len(sys.argv) != 3 or sys.argv[1] not in actions:
83         print('Usage: _python_vision.py_[tags|ocr|caption]_'
84               'URL_SAS_IMAGE')
85         sys.exit(1)
86
87     actions[sys.argv[1]](sys.argv[2])

```

```

PS C:\Users\LENOVO\Downloads\TEST_cloud> python vision.py tags "https://tp2jonathan2026.blob.core.windows.net/images/tajine%20poulet%
20citron%20confit_web.jpg?sp=r&st=2026-09-30T09:04:40Z&se=2026-09-30T22:19:40Z&spr=https&sv=2026-02-06&sr=b&sig=
Tags détectés :
- food          confidence = 0.971
- table         confidence = 0.837
- plant         confidence = 0.832
- indoor        confidence = 0.762
- bowl          confidence = 0.679
- dish          confidence = 0.674
- pot           confidence = 0.603
PS C:\Users\LENOVO\Downloads\TEST_cloud> python vision.py ocr "https://tp2jonathan2026.blob.core.windows.net/images/couverture_les_bo
uts_de_bois_de_Dieu.jpg?sp=r&st=2026-09-30T10:12:31Z&se=2026-09-30T22:27:31Z&spr=https&sv=2026-02-06&sr=b&sig=
Texte détecté :
- "OUSMANE" position = [(75, 20), (223, 19), (224, 51), (75, 52)]
- "SEMBENE" position = [(31, 52), (273, 52), (273, 102), (31, 102)]
- "Les Bouts de bois" position = [(38, 108), (260, 108), (261, 136), (38, 136)]
- "de Dieu" position = [(99, 138), (203, 137), (203, 163), (99, 163)]
- "POCKET" position = [(14, 473), (15, 406), (32, 406), (30, 473)]
PS C:\Users\LENOVO\Downloads\TEST_cloud> python vision.py caption "https://tp2jonathan2026.blob.core.windows.net/images/taine%20poul
et%20citron%20confit_web.jpg?sp=r&st=2026-09-30T09:04:40Z&se=2026-09-30T22:19:40Z&spr=https&sv=2026-02-06&sr=b&sig=
Tags détectés :
- food          confidence = 0.971
- table         confidence = 0.837
- plant         confidence = 0.832
- indoor        confidence = 0.762
- bowl          confidence = 0.679
- dish          confidence = 0.674
- pot           confidence = 0.603
Caption :
"a plate of food" confidence = 0.350

```

FIGURE 40 – Exécution de vision.py : tags (sept tags et confiances), ocr (cinq lignes avec leurs polygones) et caption (« a plate of food », confiance 0,350).

Les résultats sont identiques à ceux obtenus avec Postman (Parties V, VI et VII).

4.3 Script llm.py : LLM Foundry

Le script propose trois modes : **ask** (une question), **memory** (rejoue les deux expériences de la Partie X) et **chat** (conversation interactive où le **programme** maintient l'historique et affiche la consommation de tokens à chaque tour).

Listing 8 – Script llm.py — ask, memory et chat

```

1  """
2  TP2_-_Appels_REST_au_LLM_déployé_sur_Microsoft_Foundry.
3
4  Utilisation:
5  _python_llm.py_ask_ "Explain cloud computing in three sentences."

```

```

6  #####python_llm.py_memory#####(reproduit_les_deux_expériences_de_la
   _partie_X)
7  #####python_llm.py_chat#####(conversation_interactive_avec_
   historique)
8
9  Variables_d'environnement_:
10 #####FOUNDRY_ENDPOINT_:_URL_complète_se_terminant_par_/chat/
   completions
11 #####FOUNDRY_KEY#####:_clé_du_déploiement
12 #####FOUNDRY_MODEL#####:_nom_du_déploiement_(par_défaut_gpt-5-mini)
13 """
14
15 import os
16 import sys
17
18 import requests
19
20 ENDPOINT = os.environ["FOUNDRY_ENDPOINT"]
21 KEY = os.environ["FOUNDRY_KEY"]
22 MODEL = os.environ.get("FOUNDRY_MODEL", "gpt-5-mini")
23
24
25 def call_llm(messages):
26     """Envoie_la_liste_complète_des_messages_au_modèle_et_renvoie_
       le_JSON_de_réponse."""
27     headers = {
28         "Authorization": f"Bearer_{KEY}",
29         "Content-Type": "application/json",
30     }
31     body = {
32         "messages": messages,
33         "model": MODEL,
34         "max_completion_tokens": 4096,
35         "reasoning_effort": "medium",
36     }
37     response = requests.post(ENDPOINT, headers=headers, json=body,
                               timeout=120)
38     if not response.ok:
39         print(f"Erreur_HTTP_{response.status_code}_:_{response.text}
              ")
40         sys.exit(1)
41     return response.json()
42
43
44 def answer_and_usage(data):
45     """Extrait_le_texte_généré_et_le_nombre_de_tokens_utilisés."""
46     text = data["choices"][0]["message"]["content"]
47     usage = data["usage"]
48     return text, usage
49
50

```

```

51 def print_usage(usage):
52     reasoning = usage.get("completion_tokens_details", {}).get("
        reasoning_tokens", 0)
53     print(
54         f"[tokens]_prompt={usage['prompt_tokens']},_"
55         f"completion={usage['completion_tokens']}_("dont_
            raisonnement={reasoning}),_"
56         f"total={usage['total_tokens']}"
57     )
58
59
60 def ask(question):
61     """Partie IX: une seule question."""
62     data = call_llm([{"role": "user", "content": question}])
63     text, usage = answer_and_usage(data)
64     print(f"Assistant: {text}")
65     print_usage(usage)
66
67
68 def memory_demo():
69     """Partie X: requêtes indépendantes, puis requête avec
        historique."""
70     print("==_Expérience 1: deux requêtes séparées==")
71     ask("My favorite programming language is Python. Remember this_
        information.")
72     print()
73     ask("What is my favorite programming language?")
74
75     print("\n==_Expérience 2: historique envoyé dans une seule_
        requête==")
76     history = [
77         {"role": "user", "content": "My favorite programming_
            language is Python."},
78         {"role": "assistant", "content": "Understood."},
79         {"role": "user", "content": "What is my favorite_
            programming language?"},
80     ]
81     text, usage = answer_and_usage(call_llm(history))
82     print(f"Assistant: {text}")
83     print_usage(usage)
84
85
86 def chat():
87     """Conversation interactive: c'est le programme qui maintient_
        l'historique."""
88     history = []
89     print("Conversation avec le modèle (tape 'quit' pour arrêter).")
90     while True:
91         user_input = input("\nToi: ").strip()
92         if user_input.lower() in ("quit", "exit"):

```

```

93         break
94     if not user_input:
95         continue
96
97     history.append({"role": "user", "content": user_input})
98     text, usage = answer_and_usage(call_llm(history))
99     history.append({"role": "assistant", "content": text})
100
101     print(f"Assistant_:_{text}")
102     # prompt_tokens augmente à chaque tour, car tout l'
103     historique est renvoyé
104     print_usage(usage)
105
106 if __name__ == "__main__":
107     if len(sys.argv) >= 3 and sys.argv[1] == "ask":
108         ask("_".join(sys.argv[2:]))
109     elif len(sys.argv) == 2 and sys.argv[1] == "memory":
110         memory_demo()
111     elif len(sys.argv) == 2 and sys.argv[1] == "chat":
112         chat()
113     else:
114         print('Usage_:_python_llm.py_[ask_"question"_|_memory_|_chat]')
115         sys.exit(1)

```

```

PS C:\Users\LENOVO\Downloads\TEST_cloud> python llm.py ask "Explain cloud computing in three sentences."
Assistant : Cloud computing delivers computing resources-like servers, storage, databases, networking, and software-over the internet
so people and organizations can use them on demand without owning the physical hardware. It provides scalable, managed services main
tained by third-party providers, typically billed pay-as-you-go and offered in models such as IaaS, PaaS, and SaaS and as public, pri
vate, or hybrid deployments. By turning capital expenses into operational costs, the cloud speeds development, improves reliability a
nd global access, and lets teams focus on applications rather than infrastructure, though it also requires shared attention to securi
ty and governance.
[tokens] prompt = 13, completion = 450 (dont raisonnement = 320), total = 463
PS C:\Users\LENOVO\Downloads\TEST_cloud> python llm.py memory
=== Expérience 1 : deux requêtes séparées ===
Assistant : Got it - your favorite programming language is Python. I'll use Python examples and tailor explanations toward Python for
the rest of this conversation.

Note: I can remember this within our current chat, but I don't have a way to persistently store preferences across future, separate c
onversations. If you want, tell me any more Python-related preferences (version, frameworks, style) so I can tailor responses even mo
re.
[tokens] prompt = 17, completion = 411 (dont raisonnement = 320), total = 428

Assistant : I don't know - you haven't told me. If you want, you can:

- Tell me directly.
- Give hints (projects you work on, languages you use most, what you like about a language).
- Let me guess from a few quick questions (e.g., do you prefer concise or explicit syntax? web front end or systems programming?).

If you'd rather a one-word guess from population trends: Python. Want me to ask a few questions and try to infer?
[tokens] prompt = 13, completion = 364 (dont raisonnement = 256), total = 377

=== Expérience 2 : historique envoyé dans une seule requête ===
Assistant : Your favorite programming language is Python.
[tokens] prompt = 33, completion = 80 (dont raisonnement = 64), total = 113

```

FIGURE 41 – Modes ask (prompt = 13, completion = 450 dont 320 de raisonnement) et memory : deux requêtes séparées (« I don't know »), puis historique dans une seule requête (« Your favorite programming language is Python »).

```

PS C:\Users\LENOVO\Downloads\TEST_cloud> python llm.py chat
Conversation avec le modèle (tape 'quit' pour arrêter).

Toi : I am from Ouagadougou, Burkina Faso.
Assistant : Great - nice to meet someone from Ouagadougou! How can I help you today?

I can offer any of the following:
- Local news, events and festivals (FESPACO, SIAO, cultural life)
- Travel or itinerary planning around Ouagadougou and Burkina Faso
- Weather, climate and best times to visit (rainy season roughly May-Oct; dry/harmattan Nov-Apr)
- Practical tips: transport, markets, safety and healthcare
- Language help (French, Dioula, Mooré - translations or useful phrases)
- History, culture, food and recipes (tô, riz gras, local specialties)
- Study, job or business info and resources

Quick facts:
- Capital of Burkina Faso
- Time zone: GMT+0
- Major events: FESPACO (Pan-African film festival), SIAO (International Art & Craft Fair)
- Main airport: Ouagadougou Airport (OUA)

Tell me what you'd like - a question, plan, translation, or something else?
[tokens] prompt = 17, completion = 928 (dont raisonnement = 704), total = 945

Toi : I study cybersecurity in Rabat
Assistant : Nice - that's a great field to be in. Do you want practical study help, career advice, or local opportunities in Rabat? Below are concise, useful suggestions you can pick from.

Quick study / skill checklist
- Strong foundations: Linux, TCP/IP, networking, OS internals, web technologies (HTTP, JavaScript), basic crypto.
- Programming: Python is essential; C and some assembly help for reverse engineering/exploits.
- Hands-on tools: Nmap, Wireshark, Burp Suite, Metasploit, Ghidra (or radare2), Docker, Git.
- Labs and practice: build a home lab (VMs, Kali/Parrot, vulnerable machines), practice on TryHackMe, Hack The Box, OverTheWire, and

```

FIGURE 42 – Mode chat : début de conversation (« I am from Ouagadougou, Burkina Faso », « I study cybersecurity in Rabat »).

```

Toi : I am doing a lab on Azure AI services today.
Assistant : Nice - I can help with that. Which Azure AI service are you using in the lab (Azure OpenAI, Cognitive Services: Language/Vision/Speech/Form Recognizer, Azure ML, Cognitive Search, Semantic Kernel, Bot Service, etc.) and what language/tooling do you prefer (Python, REST/curl, JS/Node, Azure CLI)? If you tell me the lab goal I'll give precise commands or a ready-to-run snippet.

While you answer, here's a compact, practical cheat-sheet you can use for most Azure AI labs.

Quick checklist (common setup steps)
- Create the resource in the correct region with the right SKU (Azure portal / Azure CLI / ARM / Bicep / Terraform).
- Note endpoint URL and keys (or configure managed identity/Azure AD if the lab requires it).
- Confirm API version required by the SDK/REST path for the capability you're using.
- Deploy any model or indexing pipeline required (e.g., OpenAI deployment name, Cognitive Search indexer).
- Set network/security: public endpoint vs private endpoint, CORS, and firewall rules.
- Test with a single simple request before running full scenarios.

Authentication options
- API key: quick for labs - include in header: Ocp-Apim-Subscription-Key (Cognitive Services) or api-key (Azure OpenAI).
- Azure AD token / Managed Identity: more secure and recommended for production. Use azure-identity library to get credential and token.
- For Azure CLI token (quick test): az account get-access-token --resource <resource> and put Bearer <token> into Authorization header.

Typical REST pattern for Azure OpenAI (chat/completions)
- Endpoint pattern: https://<resource-name>.openai.azure.com/openai/deployments/<deployment-id>/chat/completions?api-version=<version>
- Headers: api-key: <your-key>, Content-Type: application/json
- Body (chat example):
{
  "messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "Hello"}],
  "max_tokens": 512
}
- Note: deployment name must match the model deployment you created in the portal.

```

FIGURE 43 – Mode chat : le message « I am doing a lab on Azure AI services today » reçoit une réponse détaillée.

```

Toi : Summarize everything you know about me in one sentence.
Assistant : You're from Ouagadougou, Burkina Faso, you study cybersecurity in Rabat, and you're doing a lab on Azure AI services today.
[tokens] prompt = 1930, completion = 231 (dont raisonnement = 192), total = 2161

Toi : |

```

FIGURE 44 – Mode chat : à la question « Summarize everything you know about me in one sentence », le modèle restitue l'ensemble du contexte (Ouagadougou, cybersécurité à Rabat, lab Azure AI) ; prompt = 1 930 tokens.

Ce que montre le mode chat. Le nombre de `prompt_tokens` augmente à chaque tour : il passe de 17 au premier message à 1 930 au dernier, car tout l'historique est renvoyé

à chaque requête. C'est la preuve concrète de la réponse à la question 3 de la Partie X (coût et fenêtre de contexte).

5 Supprimer les ressources Azure

Partie XII du TP.

Certains services continuent de consommer des crédits *Azure for Students* même lorsqu'ils ne sont plus utilisés. Comme toutes les ressources ont été créées dans le même groupe, on supprime le groupe `TP2-Cloud-Foundation` : la suppression efface d'un coup le compte de stockage, la ressource Computer Vision et la ressource Foundry.

6 Synthèse : utiliser un service d'IA PaaS par appels REST

Les étapes communes à tous les services d'IA d'Azure, illustrées ici par l'analyse d'images (Vision) et par le texte (LLM), sont les suivantes :

1. **Organiser** : créer un groupe de ressources qui contiendra tout le TP.
2. **Créer la ressource d'IA** : Computer Vision ou Foundry, avec sa région et son niveau tarifaire.
3. **Récupérer les identifiants** : point de terminaison et clé d'API (secrets à ne jamais publier).
4. **Préparer les données** : pour une image, la déposer dans un conteneur Blob privé et générer une SAS en lecture ; pour un LLM, préparer le tableau `messages`.
5. **Construire la requête** : méthode POST, URL de l'API (avec version et fonctionnalités), en-têtes d'authentification (`Ocp-Apim-Subscription-Key` ou `Bearer`) et `Content-Type`, corps JSON.
6. **Lire la réponse JSON** : `tags`, `texte` et `polygones` pour la vision ; `choices[0].message.content` et `usage` pour le LLM ; interpréter les codes HTTP (200, 400, 401, 404).
7. **Automatiser** en Python (`requests`), clés lues dans des variables d'environnement, historique de conversation géré par le programme.
8. **Nettoyer** en supprimant le groupe de ressources.

	Images (AI Vision)	Texte (LLM Foundry)
Authentification	En-tête <code>Ocp-Apim-Subscription-Key</code>	Autorisation <code>Bearer</code>
Données d'entrée	<code>{"url": "<SAS>"}</code>	Tableau <code>messages</code>
Résultat utile	<code>tagsResult</code> , <code>readResult</code> , <code>captionResult</code>	<code>choices[0].message.content</code>
Facturation	À l'appel (niveau gratuit F0 : quotas)	Au token (prompt + réponse + raisonnement)

TABLE 9 – Comparaison des deux usages

7 Bilan des ressources créées

Ressource	Rôle
Groupe de ressources TP2-Cloud-Foundation	Conteneur logique regroupant toutes les ressources du TP
Compte de stockage tp2jonathan2026 (Standard, LRS)	Stockage des images
Conteneur images (privé)	Contient le tajine et la couverture du livre
Ressource Computer Vision visison-jonathan-tp2 (F0)	Analyse d'image : tags, OCR, description
Ressource Foundry foundry-jonathan-tp2 (projet proj-default)	Plateforme de déploiement et d'appel des LLM
Déploiement gpt-5-mini (Global Standard)	Modèle de langage appelé par l'API Chat Completions
<i>Toutes les ressources ont été supprimées en fin de TP avec le groupe de ressources (Partie XII).</i>	

TABLE 10 – Ressources Azure impliquées dans le TP2

8 Difficultés rencontrées

Problème	Cause	Solution
Réponse « 200 Service Operational » sans résultat	URL limitée à l'endpoint, sans le chemin de l'API	Ajouter /computervision/imageanalysis:analyze
Erreur 400 sur la caption	Caption de l'API 4.0 indisponible dans les régions autorisées	Utiliser l'API Image Analysis 3.2 (Description)
Quota insuffisant pour gpt-5.4-nano	Quotas très limités des abonnements étudiants	Choisir un autre modèle
gpt-4o-mini refusé	Modèle déprécié depuis mars 2026	Déployer gpt-5-mini
Erreur « Unsupported parameter : messages »	Endpoint de la Responses API	Utiliser /openai/v1/chat/completions

TABLE 11 – Problèmes rencontrés et solutions